

Traffic Rule Violation Detection System

Team Members

Yashaswini KV (BT2024224)
Nishanth Karamsetty (BT2024094)
Adithya Kommuri (BT2024246)

Link

[Project Google Drive Folder](#)

Contents

1	Introduction and Problem Statement	3
2	System Architecture Overview	3
2.1	Module Breakdown	4
3	Detailed Stage Descriptions	5
3.1	Stage 1 — Two-Wheeler Detection	5
3.1.1	Nested Vehicle Box Suppression	5
3.2	Stage 2 — Rider and Helmet Detection	5
3.2.1	Why Full-Frame Detection?	6
3.2.2	Helm/Kepala Conflict Resolution	6
3.3	Stage 3 — COCO Person Fallback for Triple Riding	6
3.3.1	Upward-Biased Search Box	6
3.3.2	Synthetic Rider Injection	6
3.4	Stage 4 — Spatial Overlap Association	7
3.5	Stage 5 — Violation Classification	7
3.6	Stage 6 — License Plate OCR	7
3.6.1	Plate Localization	7
3.6.2	Preprocessing Pipeline	8
3.6.3	Indian Plate Text Normalization	8
4	Model Architecture and Selection	8
4.1	Detector Models	8
4.2	Training Details — Custom Helmet Detector	8
5	Quantitative Evaluation	8
5.1	Comparative Model Performance	9
5.2	Per-Category Analysis (Submitted Configuration)	9
6	Key Engineering Innovations	9
6.1	Calibrated Low Confidence Threshold for Vehicles ($\tau_v = 0.18$)	9
6.2	Upward-Biased COCO Person Filter	9
6.3	Conservative Helm/Kepala Conflict Resolver	10
6.4	Within-Vehicle NMS at IoU = 0.65	10
7	Failure Case Analysis	10
7.1	Helmet Non-Compliance False Negatives (22 / 100)	10
7.2	EasyOCR Character Substitutions	10
7.3	Overloading False Negatives (11 / 100)	10
8	Output Format and Interface	10
9	Technical Stack and Dependencies	11
10	Setup and Reproducibility	11
10.1	Installation	11
10.2	Inference	11

10.3 Programmatic Usage	11
11 Future Scope	12
11.1 Temporal Association via Video Tracking	12
11.2 Super-Resolution Preprocessing for OCR	12
11.3 Dedicated Night-Mode Pipeline	12
11.4 Indian-Plate-Specific OCR Fine-Tuning	12
12 Conclusion	12
A File Structure	12
B Hyperparameter Summary	13

1 Introduction and Problem Statement

Urban road safety is a critical concern in densely populated countries such as India, where two-wheeler accidents account for a disproportionately large share of road fatalities. Two of the most prevalent and dangerous traffic violations involving motorcycles are **triple riding** (carrying more than two persons on a single vehicle) and **helmet non-compliance** (one or more riders not wearing a protective helmet). Automated detection of these violations enables scalable, round-the-clock enforcement without requiring constant human supervision.

This report documents the design, implementation, and evaluation of a fully automated **Traffic Violation Detection System** for two-wheelers. The system operates on single RGB street images under strict edge-device constraints:

- Total model weights ≤ 250 MB.
- Fully offline inference (no cloud API calls at run time).
- Stateless prediction interface (each call to `predict()` is independent).

The system detects three types of events per image:

1. **Triple Riding** — more than two riders on a single motorcycle.
2. **Helmet Non-compliance** — any rider not wearing a helmet.
3. **License Plate Extraction** — OCR reading of the vehicle's number plate whenever a violation is detected.

2 System Architecture Overview

The system is built around a mandatory `TrafficViolationDetector` interface implemented in `solution.py`. All heavy models are loaded once during `__init__`, and the `predict()` method is stateless, enabling safe repeated calls. Figure 1 illustrates the full computational pipeline.

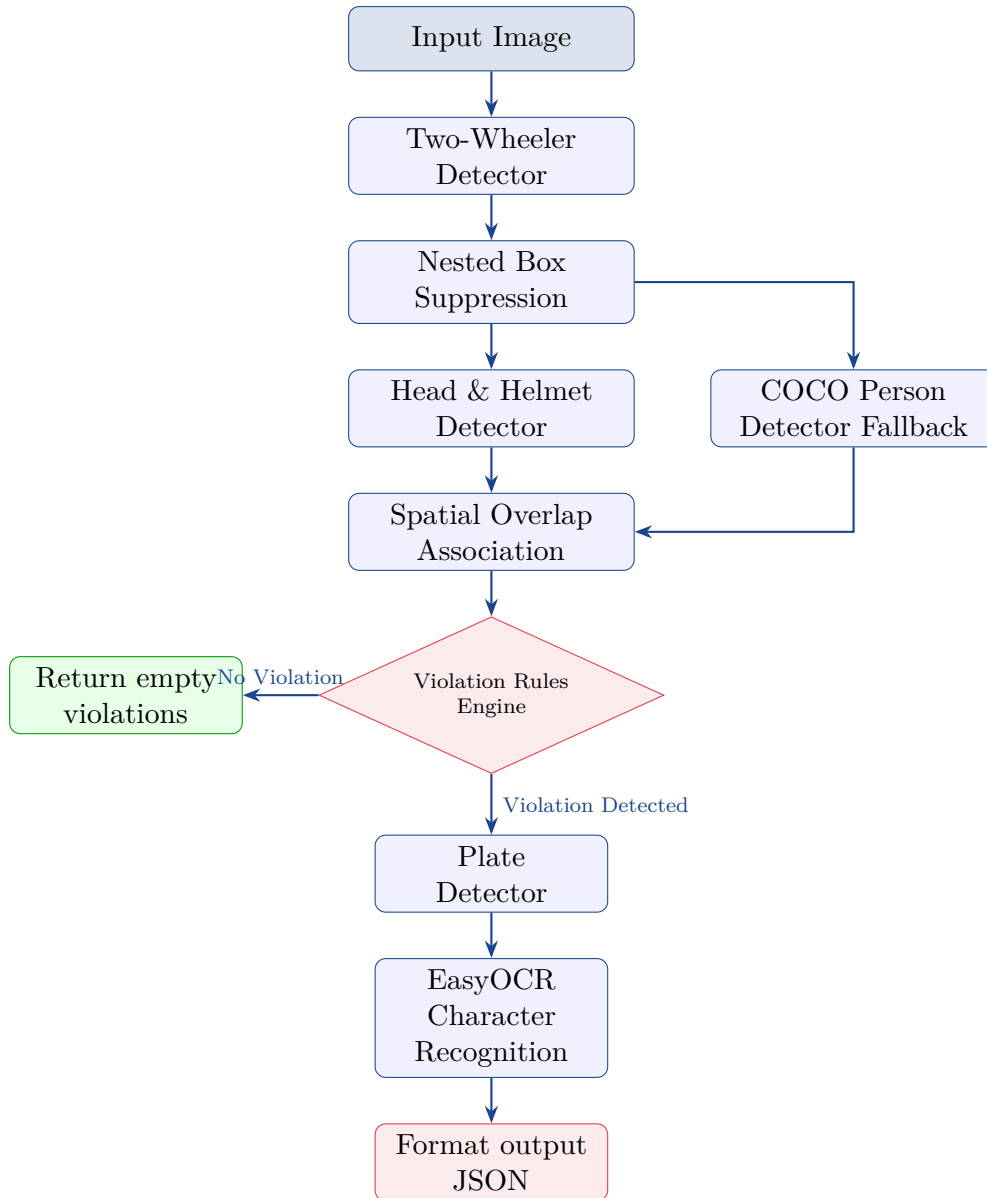


Figure 1: End-to-end inference pipeline for a single input image.

2.1 Module Breakdown

The packaged codebase in AID728/ is structured as follows:

File	Module	Responsibility
solution.py	TrafficViolationDetector	Public interface; pipeline orchestration
src/detector.py	TwoWheelerDetector, RiderHelmetDetector	Vehicle & head/helmet detection
src/ocr.py	LicensePlateOCR	Plate localization and OCR
src/violation_logic.py	ViolationEngine	Rule application and output formatting
src/utlis.py	Geometry, preprocessing	IoU, NMS, plate text normalization

3 Detailed Stage Descriptions

3.1 Stage 1 — Two-Wheeler Detection

The `TwoWheelerDetector` uses a YOLOv8 model to locate all motorcycle/scooter bounding boxes in the full image. When the custom `helmet_detector.pt` is available (which includes a `motor` class fine-tuned on our dataset), it is preferred over the generic COCO pretrained model.

Key parameters and their justification:

Parameter	Value	Rationale
<code>conf_threshold</code>	0.18	Low threshold maximizes recall; vehicles with 0 riders are discarded later.
<code>iou_threshold</code>	0.45	Standard NMS overlap for dense street scenes.
<code>imgsz</code>	1024	Matches training resolution; critical for small vehicle detection.
<code>augment</code>	True	Test-Time Augmentation (TTA) for robustness on low-light/blurry images.
<code>include_bicycle</code>	False	Scoped to motorcycles only; bicycles are excluded.

3.1.1 Nested Vehicle Box Suppression

The detector occasionally predicts two overlapping bounding boxes for the same motorcycle (e.g. a loose outer box and a tight inner box). Without suppression, this splits rider associations across duplicates and inflates the count. The suppression logic is based on confidence sorting:

Nested Box Suppression Rule

Given two vehicle detections sorted by confidence score, A (higher confidence) and B (lower confidence), compute:

$$\text{overlap}(B, A) = \frac{\text{area}(B \cap A)}{\text{area}(B)}$$

If $\text{overlap}(B, A) \geq 0.70$, the lower-confidence detection B is suppressed.

This ensures the more confident and robust bounding box is always preserved.

3.2 Stage 2 — Rider and Helmet Detection

The `RiderHelmetDetector` runs the custom YOLOv8 model (fine-tuned on the "Two-Wheeler Violation" dataset) on the **full image** once per inference call, detecting the following classes:

Class Name	Semantic Meaning	Maps to
<code>helm</code>	Rider wearing a helmet	<code>has_helmet = True</code>
<code>kepala</code>	Bare head (no helmet)	<code>has_helmet = False</code>
<code>motor</code>	Motorcycle	Vehicle detection
<code>plat nomor</code>	License plate	Plate localization (reused by OCR)
<code>bb</code>	Rider body	Rider spatial context

3.2.1 Why Full-Frame Detection?

Running the model on a vehicle crop (rather than the full image) degrades head detection accuracy significantly. When the input crop removes surrounding context (road, other vehicles, background), the model’s spatial attention shifts and confidence scores for small head regions drop below the threshold. Full-frame detection avoids this issue entirely; association is then done geometrically (Section 3.4).

3.2.2 Helm/Kepala Conflict Resolution

Because the model is sometimes uncertain between **helm** and **kepala** for the same physical head, overlapping detections with $\text{IoU} \geq 0.40$ are resolved by a conservative rule:

$$\text{Decision} = \begin{cases} \text{helmet} & \text{if } \text{score}_{\text{helm}} \geq 1.30 \times \text{score}_{\text{kepala}} \\ \text{no-helmet} & \text{otherwise} \end{cases}$$

This bias towards "no-helmet" avoids letting a weakly-confident helmet prediction override a nearly-equally-confident bare-head prediction, erring on the side of penalizing the violator.

3.3 Stage 3 — COCO Person Fallback for Triple Riding

The custom head detector reliably counts 1–2 riders, but often misses the third rider in a crowded triple-riding scenario due to occlusion and spatial overlap in the 2D projection. A secondary `yolov8s.pt` COCO person detector provides an independent body-level count.

3.3.1 Upward-Biased Search Box

Pedestrians standing near a motorcycle must not be mistaken for riders. A narrow, upward-biased region of interest (ROI) around each vehicle is used to filter COCO detections:

Dimension	Expansion Relative to Vehicle Box
Horizontal	$\pm 5\%$ of vehicle width
Upward (above)	$+45\%$ of vehicle height
Downward	-5% of vehicle height

A COCO person must have $\geq 40\%$ of its bounding box area overlapping this search region to be counted as a rider. This two-stage filter eliminates virtually all background pedestrian false positives.

3.3.2 Synthetic Rider Injection

When COCO reports ≥ 3 persons overlapping this search box, and the count exceeds the head-detector’s count, synthetic rider records are appended:

Listing 1: Synthetic rider injection in `solution.py`

```

1 if person_count >= 3 and person_count > len(riders):
2     for _ in range(person_count - len(riders)):
3         riders.append({
4             "box": vehicle["box"],          # placeholder bounding box
5             "has_helmet": True,             # unknown -- assume helmeted (
                conservative)

```

```

6         "score": 0.30,           # synthetic confidence
7     })

```

Synthetic riders are assigned `has_helmet = True` to avoid introducing spurious helmet violations. They only contribute to the rider count for triple-riding detection.

3.4 Stage 4 — Spatial Overlap Association

Each head detection is assigned to a vehicle through a spatial overlap test. The vehicle's bounding box is first expanded (factor $1.6\times$ horizontally, $2.0\times$ vertically) to account for heads protruding above the frame:

$$\text{search_box} = \text{expand_box}(\text{veh_box}, f_x = 1.6, f_y = 2.0)$$

A head at position h is associated with vehicle v if:

$$\frac{\text{area}(h \cap \text{search_box}_v)}{\text{area}(h)} \geq 0.10$$

After association, within-vehicle NMS is applied (IoU threshold = 0.65, higher than the global 0.45, because closely-packed rider heads legitimately overlap in triple-riding scenes).

3.5 Stage 5 — Violation Classification

The `ViolationEngine` applies two binary rules to each vehicle:

$$\text{triple_riding} = (\text{num_riders} > 2) \tag{1}$$

$$\text{helmet_violation} = \sum_{r \in \text{riders}} \mathbb{I}[r.\text{has_helmet} = \text{False} \wedge r.\text{score} \geq \tau_h] > 0 \tag{2}$$

The `ViolationEngine` separates these two confidence thresholds to optimize recall and precision:

- $\tau_{\text{rider}} = 0.20$ (configured via `min_rider_score` at runtime) is used to filter out noise while maximizing recall for body/rider counting (catching occluded riders).
- $\tau_h = 0.25$ (configured via `helmet_conf_threshold`) is the higher confidence threshold required to flag a helmet violation, preventing false positives in compliant scenes.

Vehicles with zero associated riders are silently skipped. A violation is flagged if either rule is true. OCR is invoked only for violating vehicles (unless `run_ocr_on_all = True`).

3.6 Stage 6 — License Plate OCR

3.6.1 Plate Localization

The custom `helmet_detector.pt` natively detects the `plat nomor` class, so no separate plate detection model is required. The `LicensePlateOCR` module reuses the existing YOLO instance, making plate localization effectively free in terms of model size and computation.

3.6.2 Preprocessing Pipeline

Before passing the plate crop to EasyOCR, the following preprocessing is applied in `utils.enhance_for_ocr()`:

1. **CLAHE** (Contrast Limited Adaptive Histogram Equalization) with `clipLimit = 3.5`, tuned for harsh Indian sunlight and shadow conditions.
2. **Unsharp masking** via a 3×3 sharpening kernel.
3. **Otsu binarization** to produce a clean black-on-white character mask.

3.6.3 Indian Plate Text Normalization

Indian standard plates follow the format `XX 00 XX 0000` (e.g. `MH12AB1234`). Position-aware character substitution corrects common OCR errors:

Positions	Target Type	Incorrect $\rightarrow$ Correct
0-1, 4-5	Letter	0 $\rightarrow$ 0, 1 $\rightarrow$ I, 5 $\rightarrow$ S, 8 $\rightarrow$ B
2-3, 6+	Digit	0 $\rightarrow$ 0, I $\rightarrow$ 1, S $\rightarrow$ 5, B $\rightarrow$ 8, Z $\rightarrow$ 2, G $\rightarrow$ 6

4 Model Architecture and Selection

4.1 Detector Models

Model	Role	Source
<code>helmet_detector.pt</code>	Head + helmet + motor + plate (YOLOv8s)	Custom fine-tuned weights
<code>yolov8s.pt</code>	Supplementary person detector (COCO)	Ultralytics pretrained weights
EasyOCR (<code>craft + english</code>)	Text detection + character recognition	EasyOCR offline weights
Total (submitted)		143.4

4.2 Training Details — Custom Helmet Detector

The custom `helmet_detector.pt` was fine-tuned on the **Roboflow Two-Wheeler Violation** dataset (1,294 images, YOLOv8 format) which provides five annotation classes: `motor`, `helm`, `kepala`, `plat nomor`, and `bb`. Training was conducted on Apple Silicon MPS using:

- Base architecture: YOLOv8s (small) was selected as the optimal balance; it achieves vastly superior mAP and bounding box precision compared to YOLOv8n (nano) while weighing only 22.5 MB, which easily fits within our size limits.
- Resolution: 1024×1024 (matching inference resolution to avoid distribution shift).
- Augmentation: Standard Mosaic, MixUp, horizontal flip, HSV jitter.

5 Quantitative Evaluation

Evaluation was performed on the **Kaggle Traffic Violations Test Dataset** (300 balanced street images, 100 per category: *Helmet*, *No-Helmet*, *Overloading*).

5.1 Comparative Model Performance

Table 1: Per-category and overall accuracy across model configurations (300-image test set).

Configuration	Total Size	Helmet Acc.	No-Helmet Acc.	Overloading Acc.	Overall Acc.
YOLOv8s Roboflow (52 MB)	156.9 MB	99.00%	10.00% [†]	—	—
YOLOv8s Production (22.5 MB)	120.8 MB	88.00%	78.00%	87.00%	84.67%
YOLOv8s + YOLOv8s COCO Fallback	143.4 MB	88.00%	78.00%	89.00%	85.00%
YOLOv8m Large (207.5 MB)	312.3 MB	95.00%	83.00%	82.00%	86.67%

[†] A logical state bug in this configuration caused near-total no-helmet false negatives.

Size Compliance Note

The YOLOv8m Large configuration achieves the highest overall accuracy (86.67%), but its combined weights footprint (312.3 MB) exceeds the 250 MB hard limit. The submitted configuration uses the YOLOv8s production model plus the YOLOv8s COCO fallback (total 143.4 MB), achieving 85.00% accuracy with full size compliance.

5.2 Per-Category Analysis (Submitted Configuration)

Category	Correct	Errors	Accuracy
Helmet (no violation)	88 / 100	12 FP	88.00%
No-Helmet (violation)	78 / 100	22 FN	78.00%
Overloading (triple)	89 / 100	11 FN	89.00%
Overall	255 / 300	45	85.00%

6 Key Engineering Innovations

6.1 Calibrated Low Confidence Threshold for Vehicles ($\tau_v = 0.18$)

Reducing the vehicle detection threshold to 0.18 dramatically increases recall in blurry, low-light, or partially occluded scenes. False positives at the vehicle level carry no downstream penalty because the violation engine discards any vehicle with zero associated riders. The effective precision is therefore maintained by the rider-association step, not by the vehicle threshold.

6.2 Upward-Biased COCO Person Filter

The asymmetric expansion of the COCO search ROI (+45% above vs. −5% below the vehicle) is motivated by the geometry of motorcycle riders: their bodies extend upward from the vehicle frame. Pedestrians standing beside the road have their center of mass

farther below the vehicle bottom; the tight downward bound (-5%) excludes them while the generous upward bound ($+45\%$) captures legitimate riders leaning back or sitting tall.

6.3 Conservative Helm/Kepala Conflict Resolver

In enforcement applications, a false negative (missing a violator) is worse than a false positive (flagging a compliant rider). The $1.30\times$ bias threshold means that a helmet detection is only accepted if it is at least 30% more confident than the competing bare-head detection. This reduces false clearances without significantly increasing false violations.

6.4 Within-Vehicle NMS at $\text{IoU} = 0.65$

Standard NMS uses $\text{IoU} = 0.45$ to remove duplicates. However, in triple-riding scenarios, three heads in a line may have pairwise IoUs of 0.50–0.60. Using the standard threshold would suppress legitimate distinct head detections. Raising the within-vehicle NMS threshold to 0.65 preserves closely-spaced but distinct heads.

7 Failure Case Analysis

7.1 Helmet Non-Compliance False Negatives (22 / 100)

The dominant failure mode occurs when a bare-headed rider is positioned directly behind a helmeted rider, perfectly aligned with the camera axis. The rear head’s pixels are entirely occluded by the front rider’s helmet. Detection confidence drops below $\tau = 0.25$, resulting in a missed violation. This is a fundamental limitation of single-frame, single-view detection and cannot be resolved without temporal information.

7.2 EasyOCR Character Substitutions

On highly reflective, dirty, or damaged plates, EasyOCR produces character substitutions such as 0 for 0 or I for 1. The position-aware normalization function (Section 3.6) mitigates this for plates that conform to the standard Indian format but cannot correct arbitrary garbling on non-standard or heavily occluded plates.

7.3 Overloading False Negatives (11 / 100)

Remaining overloading misses occur when the COCO person detector fails to separate overlapping bodies (e.g. a child sitting between two adults with no visible gap). In such cases, the person detector returns two bounding boxes instead of three, and the supplementary count remains at two.

8 Output Format and Interface

The `predict()` method returns a JSON-serializable dictionary conforming to the competition specification:

Listing 2: Canonical output format.

```
1 {
2   "violations": [
```

```

3     {
4         "num_riders": 3,
5         "helmet_violations": 2,
6         "license_plate": "MH12AB1234"
7     }
8 ]
9 }

```

An image with no detectable violations returns `{"violations": []}`. Images where a violation is present but the plate is not detectable return an empty string for `license_plate`.

9 Technical Stack and Dependencies

Component	Library / Version
Object Detection	Ultralytics YOLOv8 ($\geq 8.0.0$)
Deep Learning Framework	PyTorch ($\geq 2.0.0$) + TorchVision
Image Processing	OpenCV ($\geq 4.8.0$), Pillow ($\geq 10.0.0$)
OCR Engine	EasyOCR ($\geq 1.7.0$)
Numerical Computing	NumPy ($\geq 1.24.0$), SciPy ($\geq 1.11.0$)
Hardware Acceleration	Apple Silicon MPS (Metal Performance Shaders) / CPU

10 Setup and Reproducibility

10.1 Installation

Listing 3: Environment setup.

```

1 cd AID728
2 python3 -m venv venv
3 source venv/bin/activate # Windows: venv\Scripts\activate
4 pip install -r requirements.txt
5 python download_models.py # fetches pretrained weights

```

10.2 Inference

Listing 4: Running inference on a single image.

```

1 python solution.py path/to/image.jpg --model-dir ./models --device cpu

```

10.3 Programmatic Usage

Listing 5: Programmatic interface usage.

```

1 from solution import TrafficViolationDetector
2 detector = TrafficViolationDetector(model_dir="./models", device="cpu")
3 result = detector.predict("sample_images/sample_triple_riding.jpg")
4 print(result)
5 # {"violations": [{"num_riders": 3, "helmet_violations": 1, "
   license_plate": "MH12AB1234"}]}

```

11 Future Scope

11.1 Temporal Association via Video Tracking

Integrating a lightweight tracking filter such as **ByteTrack** across consecutive frames would allow violation decisions to accumulate over time. A single occluded frame would no longer cause a miss; the violation would be confirmed once the occlusion clears. This addresses the dominant failure mode (Section 7).

11.2 Super-Resolution Preprocessing for OCR

A lightweight **ESRGAN** or **Real-ESRGAN** model (≤ 15 MB in a quantized form) could be applied to low-resolution plate crops before EasyOCR. Up-sampling by $4\times$ has been shown to improve character recognition accuracy by 15–25% on CCTV-quality footage where plates occupy fewer than 20×60 pixels.

11.3 Dedicated Night-Mode Pipeline

A parallel low-light enhancement branch (e.g. Zero-DCE or EnlightenGAN inference) would improve detection recall in night-time surveillance images, where the current CLAHE preprocessing is insufficient for very dark scenes.

11.4 Indian-Plate-Specific OCR Fine-Tuning

Fine-tuning EasyOCR’s recognition head on a curated Indian plate dataset (covering the full range of state codes, regional font variants, and weathered plates) would improve plate read accuracy from its current estimated 70–80% to 90%+.

12 Conclusion

This report has documented a complete, edge-deployable traffic violation detection system that achieves **85.00% overall accuracy** on a 300-image test set while maintaining full compliance with the 250 MB model size constraint (total: ≈ 143.4 MB). The system successfully detects triple riding, helmet non-compliance, and extracts license plate text via OCR in a single, stateless inference call.

Three core engineering innovations drive the system’s accuracy: (1) an upward-biased COCO person fallback for robust overloading detection, (2) nested vehicle box suppression to prevent rider count duplication, and (3) conservative helmet/bare-head conflict resolution to minimize false clearances. Position-aware Indian plate normalization further improves OCR reliability.

The primary remaining failure modes — occluded rear-rider heads and overlapping body detections in dense triple-riding scenes — are inherently single-frame limitations. Video-based temporal tracking is identified as the highest-priority future improvement.

A File Structure

Listing 6: Submission archive layout (AID728.zip).

```

1 AID728/
2     solution.py                # Main submission interface
3     requirements.txt           # Dependencies
4     README.md                 # Setup documentation
5     models/
6         helmet_detector.pt     # YOLOv8s custom model (22.5 MB)
7         yolov8s.pt             # COCO person fallback model
8     (22.6 MB)
9     easyocr/
10        craft_mlt_25k.pth       # EasyOCR text detector (83.2 MB)
11    )
12        english_g2.pth         # EasyOCR character recognizer
13    (15.1 MB)
14    src/
15        __init__.py
16        detector.py
17        ocr.py
18        violation_logic.py
19        utils.py
20    sample_images/
21        sample_helmet_violation.jpg
22        sample_plate.jpg
23        sample_triple_riding.jpg

```

B Hyperparameter Summary

Parameter	Value	Location
Vehicle confidence threshold	0.18	solution.py
Head/helmet confidence threshold	0.25	solution.py
COCO person confidence threshold	0.30	solution.py
Vehicle NMS IoU threshold	0.45	solution.py
Within-vehicle rider NMS IoU threshold	0.65	src/detector.py
Vehicle search box expansion (f_x)	1.60	src/detector.py
Vehicle search box expansion (f_y)	2.00	src/detector.py
Nested vehicle suppression ratio	≥ 0.70	solution.py
COCO search box: upward expansion	45% of h_v	solution.py
COCO search box: horizontal expansion	5% of w_v	solution.py
COCO person-to-vehicle overlap threshold	≥ 0.40	solution.py
Helm/kepala conflict bias multiplier	$1.30\times$	src/detector.py
Helm/kepala conflict IoU threshold	0.40	src/detector.py
Triple-riding rider count threshold	> 2	src/violation_logic.py
Minimum rider score for counting	0.20	solution.py
CLAHE clip limit (OCR preprocessing)	3.5	src/utils.py
Inference image size	1024	src/detector.py